

Fuse Open Design Decisions

Sep 25, 2026 · @Kamal Koushik Duppalapudi

While fixing bugs in Fuse, I found 12 things that aren't simple fixes — each one involves a trade-off, and needs a decision from you before any code changes.

The 12 things that need a decision

#	Topic	What's going on	What we could do
1	Spreading work across threads more smartly	We built code that can balance work across threads and react to network conditions on the fly, but the actual sending/receiving logic doesn't use it yet — it still splits work up in a fixed way decided at the start. Switching over would change how the core of the program runs, not just fix a bug.	Turn it on for real (bigger change, real upside), leave things as they are for now, or turn on just the parts with a clear benefit and leave the rest for later.
2	No support for newer internet addresses (IPv6)	Fuse only understands older-style addresses (like 192.168.1.1). It can't talk to a device that only has a newer-style address. Every networking function in the code assumes the old style, so adding support touches almost everything.	Add support for both address styles now, or leave it as-is until it's actually needed.

#	Topic	What's going on	What we could do
3	Keeping a spare copy of every unsent chunk	While sending data, Fuse keeps a backup copy of each recent chunk in memory in case it needs to resend it — about 4MB per connection. We could save that memory by pointing at the original data instead of copying it. That's safe in one part of the code, but risky in another: if the app deletes its original data too soon, Fuse could end up reading memory that's no longer valid, which can crash the program.	Leave the memory cost as it is (it's not huge), fix it only where it's safe today, or change the rules for how long the app must keep its data around so it's safe everywhere.
4	Sending or receiving a file loads the whole thing into memory	Right now, sending a 10GB file needs about 10GB of free memory, and receiving one can briefly need up to 20GB. Fixing this means reading and writing the file in smaller pieces instead of all at once.	Build the smaller-pieces version now, or keep documenting the memory requirement for large files as a known limit.
5	Two small limits built into how data is packed for sending	How many messages can be "in flight" at once is capped at 255, and each piece of data sent doesn't carry a unique ID for its connection. Both would need a change to the actual format data is packed in for sending — and we just changed that format for a security fix, so making another change right away felt risky to review all at once.	Bundle both changes into the next format update, or wait until another format change is needed anyway and do them together.

#	Topic	What's going on	What we could do
6	Some short "status" messages aren't fully protected	When encryption is on, the actual data is locked down, but short status messages — like "got it," "please resend this," "closing the connection" — aren't. We already fixed the most dangerous version of this (an attacker could freeze a connection). Closing the rest of the gap means encrypting these status messages too, a bigger change.	Encrypt the status messages too (more overhead, closes the gap), or accept the small remaining risk — an attacker would already need to know exactly which connection to target.
7	Retried connection attempts can leave a "ghost" behind	If a connection attempt is retried — say, because a network packet got lost — the server can end up creating two half-finished connections instead of recognizing it's the same attempt. This wastes a bit of memory and a thread per leftover, but it's no longer a security problem (that part is already fixed).	Make retries properly reuse the first attempt (fiddly to get right), add a cleanup process that closes connections that never finish setting up, or leave it — it's limited by how many times someone can realistically retry.
8	Without a password, the client can't verify who it's talking to	If you run Fuse without a shared password, there's no way for the client to check it's really talking to the right server — it'll accept a reply from anyone. This is just how unencrypted mode works, not a bug, but it should be written down clearly so nobody is surprised by it.	Add a basic sanity check that catches accidents or misconfiguration (not a real attacker), or just document clearly that unencrypted mode should only be used on a trusted network.

#	Topic	What's going on	What we could do
9	No warning when you turn on a setting that slows things down	There's a setting that pins worker threads to specific CPU cores. Our own tests show this makes things about 6% slower, but nothing warns you if you turn it on anyway — because the library currently has no way to print messages or log anything at all. Adding one warning means either printing straight to the screen (which nothing else in the library does) or building a whole logging system just for this.	Build a small logging system now, leave the warning in the documentation only (as today), or have the setting hand back a value the calling program can print itself.
10	An advanced speed optimization for big servers is unverified	There's a feature that places work near the right part of the machine on large, multi-processor servers, and it's written to safely do nothing on machines that don't support it. What we can't tell just from reading the code is whether it was ever actually tested and measured on real hardware.	Run a real test on the right hardware and document the result, or mark it as "built but unverified" for now.

#	Topic	What's going on	What we could do
11	One function does too many jobs at once	The function that handles sending data is about 280 lines long and handles the handshake, packing data, handling confirmations, timeouts, and more, all in one place. Splitting it into smaller, focused pieces would make it easier to read and maintain.	Split it up now while it's fresh in mind, or wait a while — a lot of careful, tested changes just landed in that exact code, and touching it again immediately adds risk.
12	A scanning tool flagged 195 pieces of code that look unused	An automated tool that maps how the code connects together flagged 195 pieces that look disconnected from everything else. This isn't something to fix in code — someone needs to look and decide whether it's genuinely unused code we can delete, or whether the tool just made a mistake.	Review the list and delete what's actually unused, check whether the tool itself is misreading the code, or leave it alone since it's not clearly one or the other.